Laborator #1 : suport online (Îndrumar de laborator)

Sumar:

I) Lucrul la linia de comandă în Linux, cu comenzi simple (partea întâia)

I.1) Recap: Reamintirea unor cunoștințe dobândite în lecția practică despre comenzi simple

I.2) Exemplificări: Exemple de utilizare a unor comenzi simple (exerciții rezolvate)

I) Lucrul la linia de comandă în Linux, cu comenzi simple (partea întâia) :

Prolog: înainte de a începe studiul comenzilor disponibile pentru lucrul la linia de comandă în Linux, revedeți prezentarea introductivă despre sistemele de operare din familia UNIX, și în particular despre Linux, disponibilă aici.

Recomandare: Show / Hide the recommendation

Nu vă rezumați doar la consultarea prezentării PPT indicate mai sus; citiți și subcapitolele corespunzătoare din cartea recomandată pentru componenta practică: Cap. 1, §1.1 și Cap. 1, §1.2.

I.1) Reamintirea unor cunoștințe dobândite în lecția practică despre comenzi simple :

Informații despre comenzi simple pentru lucrul la linia de comandă în Linux:

(Re)citiți prezentarea cu lecția practică despre comenzi de bază și sisteme de fișiere în Linux, disponibilă aici.

Recomandare: Show / Hide the recommendation

Nu vă rezumați doar la consultarea prezentării PPT indicate mai sus; citiți și subcapitolele corespunzătoare din cartea recomandată pentru componenta practică: Cap. 2, §2.1 și Cap. 2, §2.2.

I.2) Exemple de utilizare a unor comenzi simple (exerciții rezolvate) :

1. [Accessing the man pages] **Exemplificarea comenzilor pentru consultarea paginilor de manual**. Paginile de manual conțin documentația despre comenzile externe disponibile în Linux, precum și despre funcțiile C din API-ul POSIX și cele din biblioteca standard de C, plus alte documentații, organizate pe secțiuni.

- **i)** Comanda `whatis` ne permite să aflăm dacă există vreo comandă sau vreo funcție cu un anumit nume specificat.
  Spre exemplu, dacă apelăm comanda cu argumentul "write", în felul următor:
  prompt> `whatis write`
  atunci outputul afișat pe ecran de această comandă va fi următorul:

  ```
  write (1)            - send a message to another user
  write (2)            - write to a file descriptor
  ```

  Prin urmare, astfel aflăm că există o comandă cu numele write, deoarece "(1)" indică secțiunea în care se găsește prima semnificație găsită pentru write, și cunoaștem faptul că secțiunea 1 a paginilor de manual conține documentația despre comenzile uzuale (doar cele externe) accesibile utilizatorilor obișnuiți. Iar a doua semnificație găsită pentru write este în secțiunea 2, ceea ce ne indică faptul că există o funcție în API-ul POSIX cu numele write (deoarece cunoaștem faptul că secțiunea 2 a paginilor de manual conține documentația despre funcțiile C din API-ul POSIX). În plus, scurta descriere ce însoțește fiecare dintre cele două semnificații găsite pentru write, ne dă o indicație asupra funcționalității pe care-o oferă comanda write, respectiv funcția write.
  Iată și un alt exemplu:
  prompt> `whatis mkdir`

  ```
  mkdir (1)            - make directories
  mkdir (2)            - create a directory
  ```

  Astfel aflăm că există o comandă uzuală numită mkdir, precum și o funcție cu același nume în API-ul POSIX. În plus, ni se dă și o indicație asupra funcționalității oferite de către acestea, prin scurta descriere ce le însoțește.

- **ii)** Comanda `man` ne permite să consultăm pagina de manual, dintr-o anumită secțiune, pentru un anumit cuvânt, reprezentând numele unei comenzi sau al unei funcții.
  Spre exemplu, dacă apelăm comanda cu argumentul "mkdir", în felul următor:
  prompt> `man 1 mkdir`
  (sau putem tasta doar `man mkdir` , caz în care se va selecta prima secțiune de manual ce conține acel cuvânt, adică în cazul de față secțiunea 1),
  atunci outputul afișat pe ecran de această comandă este de forma următoare: Show / Hide the output
  Din acest output, putem observa cum arată structura unei pagini de manual ce documentează o comandă, și anume are următoarele "paragrafe":
  - NAME -- conține o linie de text, cu numele comenzii și o scurtă descriere a funcționalității oferite (este chiar descrierea afișată de comanda `whatis`);
  - SYNOPSIS -- descrie sintaxa comenzii, adică parametrii liniei de comandă prin care se poate apela comanda respectivă;
  - DESCRIPTION -- descrie semnificația parametrilor liniei de comandă, atât cei de tip opțiuni, cât și a celor reprezentând argumentele specifice ale comenzii, în cazul de față numele directoarelor ce se vor crea;
  - SEE ALSO -- enumeră alte pagini de manual ce conțin documentația unor comenzi sau funcții ce oferă funcționalități corelate cu cea a comenzii de față.
  - Mai puteți observa și alte "paragrafe", dar acestea au informații mai puțin relevante.

  Observație: comanda `man` este de fapt un *pipeline* (un lanț de comenzi), în care comenzii `less` îi revine sarcina efectivă de afișare pe ecran a conținutului generat de restul *pipeline*-ului. Ca atare, pentru a interacționa cu conținutul afișat de o comandă `man`, putem folosi toate tastele ce au asociate 'comenzi de navigare', descrise în documentația comenzii `less`.

  Voi aminti aici doar câteva dintre aceste 'comenzi de navigare' disponibile în comanda `less`:

i) Pentru a ieși din modul de afișare a unei pagini de manual, se apasă tasta Q.
ii) Pentru a naviga "în sus și în jos" atunci când informațiile din pagina respectivă "ocupă" mai mult de un ecran, se folosesc fie tastele Up (săgeată sus) și Down (săgeată jos) (pentru a trece la linia următoare), fie tastele PgUp/PgDn (pentru a trece la ecranul următor). Alte taste pentru navigare mai sunt: tasta g (sau tasta <) pentru a reveni la începutul fișierului afișat, respectiv tasta G (sau tasta >) pentru a ajunge la finalul fișierului.
iii) Putem căuta cuvinte (inclusiv șabloane gen expresii regulate) și putem naviga la prima apariție a cuvântului căutat, apăsând tasta '/' urmată imediat de cuvântul căutat plus ENTER, cum ar fi de exemplu numele unui "paragraf", sau orice altceva; dacă sunt mai multe apariții ale acelui cuvânt în pagina de manual, vor fi evidențiate toate aparițiile cuvântului în text, prin schimbarea culorii de fundal a fiecărei apariții. Și putem naviga de la o apariție la alta a cuvântului căutat în text apăsând în mod repetat tasta n. Respectiv, cu tasta N apăsată în mod repetat, putem să ne întoarcem înapoi de la o apariție la alta.

Recomandare: pentru informații suplimentare referitoare la alte facilități de navigare/căutare/etc. prin pagina de manual afișată, consultați documentația comenzii `less`. Sau puteți apăsa tasta h, în timp ce este activă comanda `less` (i.e., în timp ce vizualizați un text cu ajutorul comenzii `less`), pentru a obține un help cu toate tastele și 'comenzile' asociate lor.

Pentru a vedea și care este structura unei pagini ce documentează o funcție C, vom da drept exemplu:
prompt> `man 2 mkdir`
(aici nu putem tasta doar `man mkdir` , deoarece s-ar selecta prima secțiune de manual ce conține acel cuvânt, adică în cazul de față secțiunea 1, care nu reprezintă ceea ce ne interesează),
atunci outputul afișat pe ecran de această comandă este de forma următoare: Show / Hide the output
Din acest output, putem observa cum arată structura unei pagini de manual ce documentează o funcție C din API-ul POSIX (similar arată și structura paginilor ce documentează funcții C din biblioteca standard de C), și anume are următoarele "paragrafe":

- NAME -- conține o linie de text, cu numele funcției și o scurtă descriere a funcționalității implementate în acea funcție (este chiar descrierea afișată de comanda `whatis`);
- SYNOPSIS -- descrie sintaxa funcției, adică prototipul funcției, plus lista fișierelor header care trebuie incluse pentru a se putea apela funcția respectivă;
- DESCRIPTION -- descrie semnificația argumentelor funcției, precum și funcționalitatea implementată în acea funcție;
- RETURN VALUE -- descrie semnificația valorii returnate de funcție, inclusiv în caz de eroare;
- ERRORS -- descrie lista unor constante simbolice, a căror valoare numerică este stocată în variabila de sistem `errno` în caz de terminare cu eroare a apelului funcției, precum și motivele ce cauzează terminarea cu eroare, asociate acelor constante simbolice;
- NOTES -- observații suplimentare despre funcția respectivă;
- SEE ALSO -- enumeră alte pagini de manual ce conțin documentația unor comenzi sau funcții ce oferă funcționalități corelate cu cea a funcției de față.
- Mai puteți observa și alte "paragrafe", dar acestea au informații mai puțin relevante.

Explicații ale câtorva concepte de bază (ce vor fi reîntâlnite și în cursurile teoretice) :
Biblioteca standard de C, referită pe scurt prin denumirea `libc`:

Aceasta este o bibliotecă de funcții standard ce pot fi apelate din toate programele scrise în limbajul C (și uneori din programe scrise în alte limbaje de programare), indiferent pentru ce platformă sunt compilate acele programe, deoarece biblioteca `libc` are implementări pentru toate platformele pentru care avem compilatoare de C (e.g.,

Windows, Linux și orice alt sistem UNIX). Pentru fiecare platformă, pot exista chiar mai multe implementări ale bibliotecii `libc`.
Spre exemplu, pentru platforma Linux cea mai populară este implementarea GNU C Library, referită pe scurt prin denumirea `glibc`. Pentru a afla mai multe informații despre implementările bibliotecii standard de C pentru platforma Linux, puteți folosi comanda `man 7 libc` pe sistemul Linux pe care lucrați, sau puteți accesa varianta online a acestei documentații, disponibilă aici.

Apelurile de sistem furnizate de un sistem de operare:
Acestea sunt rutine implementate în nucleul unui sistem de operare ce implementează diverse servicii oferite de acel sistem de operare (și ele sunt executate în modul privilegiat al procesorului -- așa-numitul *kernel-mode*). Programele ce rulează pe acel sistem de operare (în modul neprivilegiat al procesorului -- așa-numitul *user-mode*) pot beneficia de serviciile respective invocând rutina (apelul de sistem) ce implementează serviciul dorit. Modul exact de invocare a unui apel de sistem depinde de sistemul de operare respectiv.
Astfel, în cazul platformei Linux, apelurile de sistem sunt invocate în mod indirect, prin intermediul unor funcții *wrapper* definite în biblioteca `glibc`, care execută unele operații suplimentare înainte de a comuta procesorul în modul privilegiat pentru a executa apelul de sistem propriu-zis. Pentru a afla mai multe informații despre apelurile de sistem furnizate de nucleul Linux (și funcțiile *wrapper* asociate lor), puteți folosi comanda `man 2 syscalls` pe sistemul Linux pe care lucrați, sau puteți accesa varianta online a acestei documentații, disponibilă aici.
Setul de apeluri de sistem furnizat de un sistem de operare este denumit API-ul (i.e., interfața de programare a aplicațiilor) acelui sistem.

Standarde ale API-ului (i.e., setul de apeluri de sistem) furnizat de un sistem de operare din familia UNIX:
Acestea sunt specificații ale prototipurilor funcțiilor ce formează acel API.
Astfel, în structura unei pagini de manual precum cea exemplificată mai sus, "paragraful" CONFORMING TO se referă la standardul (sau standardele) ce conține ca specificație funcția respectivă cu prototipul de acea formă.
Pe parcursul dezvoltării sistemelor UNIX, au existat și alte standarde apărute înaintea standardului POSIX, și toate aceste standarde au cunoscut diverse versiuni de elaborare a specificațiilor respective. Pentru a afla mai multe informații despre standardele existente puteți folosi comanda `man 7 standards` pe sistemul Linux pe care lucrați, sau puteți accesa varianta online a acestei documentații, disponibilă aici.

- **iii)** Comanda `which` ne permite să aflăm calea absolută a fișierului ce conține o comandă externă, dacă aceasta este accesibilă (prin intermediul variabilei de mediu `PATH`, despre care vom discuta mai târziu).
  Spre exemplu, dacă apelăm comanda cu argumentul "mkdir", în felul următor:
  prompt> `which mkdir`
  atunci outputul afișat pe ecran de această comandă este următorul:

  ```
  /bin/mkdir
  ```

  În concluzie, programul executabil ce implementează comanda mkdir este localizat la calea absolută `/bin/mkdir`.

- **iv)** Comanda `whereis` ne permite să aflăm calea absolută a fișierului executabil ce conține o comandă externă, cea a codului sursă a acelei comenzi (dacă poate fi localizat pe sistemul

respectiv), precum și cele ale paginilor de documentație pentru semnificațiile posibile ale numelui comenzii.
Spre exemplu, dacă apelăm comanda cu argumentul "mkdir", în felul următor:
prompt> `whereis mkdir`
atunci outputul afișat pe ecran de această comandă este următorul:

```
mkdir: /bin/mkdir /usr/share/man/man2/mkdir.2.gz /usr/share/man/man1/mkdir.1.gz
```

În acest caz, `/bin/mkdir` este calea absolută a programului executabil ce implementează comanda mkdir, codul sursă nu a fost găsit, iar celelalte două căi absolute descriu arhivele în format gzip ce stochează pe disc conținutul celor două pagini de manual pentru mkdir (comanda mkdir, din secțiunea 1, și respectiv funcția mkdir, din secțiunea 2 a manualului).

- **v)** Comanda `apropos` ne vine în ajutor atunci când am uitat numele vreunei comenzi sau funcții de care avem nevoie, permițându-ne să căutăm apariițiile unui cuvânt (sau a mai multor cuvinte) prin descrierile sumare cuprinse în "paragraful" NAME ale tuturor paginilor de manual.
  Succesul căutării cu comanda `apropos` va depinde esențial de gradul de relevanță al cuvintelor pe care le alegem pentru căutare, similar cu ceea ce se întâmplă în cazul căutărilor pe motorul de căutare Google, sau altele asemănătoare.
  Spre exemplu, dacă apelăm comanda cu verbul a scrie ("write"), în felul următor:
  prompt> `apropos write`
  atunci outputul afișat pe ecran de această comandă va conține o listă numeroasă de rezultate. Apoi va trebui să inspectăm vizual fiecare rezultat din listă și să vedem dacă se referă la comanda sau funcția care ne interesa. Eventual, mai putem restrânge cazurile analizate, uitându-ne doar la rezultatele ce aparțin de secțiunea de manual ce ne interesează (1 - pentru comenzi, 2 - pentru funcții din API-ul POSIX, ș.a.m.d.).

  Căutarea ar putea fi mai eficientă dacă, drept cuvinte ce ar putea apare în descrierea funcționalității căutate, am folosi cuvinte ce sunt mai puțin generale, nu cum este cazul verbului a scrie ("write").
  Spre exemplu, să spunem că am uitat cum se numește comanda care copie fișiere. Am putea căuta cuvintele "copy" și "file", astfel:
  prompt> `apropos copy file`
  Însă, și în acest caz lista de rezultate afișată va fi foarte mare. Observăm că fiecare rezultat afișat conține cel puțin unul dintre cele două cuvinte căutate, dar nu neapărat pe amândouă.
  Pentru a forța comanda `apropos` să returneze ca rezultate doar paginile de manual ce conțin simultan ambele cuvinte în descrierea sumară din "paragraful" NAME, putem folosi opțiunea -a (sau --and), astfel:
  prompt> `apropos -a copy file`
  În acest caz, lista de rezultate obținută va fi mult mai scurtă: Show / Hide the output
  Acest lucru ne va ușura apoi inspecția vizuală a fiecărui rezultat din listă pe care o vom face pentru a vedea dacă se referă la comanda sau funcția care ne interesa. Bineînțeles, și în acest caz putem să restrângem și mai mult cazurile analizate, uitându-ne doar la rezultatele ce aparțin de secțiunea de manual ce ne interesează.

Pentru a încheia acest prim "exercițiu rezolvat", voi enumera în cele ce urmează cele 8 secțiuni în care sunt organizate paginile de manual.

1. **Secțiunea 1** a manualului descrie comenzile și instrumentele disponibile utilizatorilor sistemului -- de exemplu, comenzi de manipulare a fișierelor, shell-uri, compilatoare, browsere web, vizualizatoare și editori de fișiere și imagini și așa mai departe. Pentru a afla mai multe informații despre secțiunea 1, puteți folosi comanda `man 1 intro` pe sistemul Linux pe care lucrați, sau puteți accesa varianta online a acestei documentații, disponibilă aici.

2. **Secțiunea 2** a manualului descrie apelurile de sistem oferite de platforma Linux. Pentru a afla mai multe informații despre secțiunea 2, puteți folosi comanda `man 2 intro` pe sistemul Linux pe care lucrați, sau puteți accesa varianta online a acestei documentații, disponibilă aici.
3. **Secțiunea 3** a manualului descrie toate funcțiile din biblioteca `libc`, cu excepția funcțiilor *wrapper* ale apelurilor de sistem (ce sunt descrise în secțiunea 2), plus funcții din alte biblioteci. Pentru a afla mai multe informații despre secțiunea , puteți folosi comanda `man 3 intro` pe sistemul Linux pe care lucrați, sau puteți accesa varianta online a acestei documentații, disponibilă aici.
4. **Secțiunea 4** a manualului descrie fișierele speciale (*device files*, i.e. `/dev/*`). Pentru a afla mai multe informații despre secțiunea 4, puteți folosi comanda `man 4 intro` pe sistemul Linux pe care lucrați, sau puteți accesa varianta online a acestei documentații, disponibilă aici.
5. **Secțiunea 5** a manualului descrie diverse formate de fișiere, precum și diverse tipuri de sisteme de fișiere. Pentru a afla mai multe informații despre secțiunea 5, puteți folosi comanda `man 5 intro` pe sistemul Linux pe care lucrați, sau puteți accesa varianta online a acestei documentații, disponibilă aici.
6. **Secțiunea 6** a manualului descrie jocurile și micile programe amuzante disponibile pe sistem. Pentru a afla mai multe informații despre secțiunea 6, puteți folosi comanda `man 6 intro` pe sistemul Linux pe care lucrați, sau puteți accesa varianta online a acestei documentații, disponibilă aici.
7. **Secțiunea 7** a manualului oferă o privire de ansamblu asupra a diverse subiecte și descrie convențiile și protocoalele, standardele pentru seturile de caractere, aspectul standard al sistemului de fișiere și alte lucruri diverse. Pentru a afla mai multe informații despre secțiunea 7, puteți folosi comanda `man 7 intro` pe sistemul Linux pe care lucrați, sau puteți accesa varianta online a acestei documentații, disponibilă aici.
8. **Secțiunea 8** a manualului descrie comenzi utilizabile numai de către superutilizator, cum ar fi comenzile de administrare a sistemului, demonii și comenzile legate de hardware. Pentru a afla mai multe informații despre secțiunea 8, puteți folosi comanda `man 8 intro` pe sistemul Linux pe care lucrați, sau puteți accesa varianta online a acestei documentații, disponibilă aici.

2. [Basic file operations #1] **Exemplificare creare diferite tipuri de fișiere.**

Scrieți comenzile care creează următoarele tipuri de fișiere:

- **i)** un subdirector numit `d1`, în directorul dvs. acasă; două subdirectoare `d2` și `d3`, în subdirectorul `d1` creat anterior; un subdirector `d4`, în subdirectorul `d2` creat anterior.

  Recomandare: mai întâi, pentru a înțelege exemplele de rezolvare de mai jos, (re)citiți documentațiile comenzilor `mkdir` și `ls`, disponibile pe sistemul Linux pe care lucrați folosind comenzile `man 1 mkdir` și respectiv `man 1 ls` , sau accesați variantele online ale acestor documentații, disponibile aici și respectiv aici. Cât despre comanda `cd`, aceasta fiind comandă internă, puteți accesa documentația ei pe sistemul Linux pe care lucrați folosind comanda `help cd` .

  Show / Hide some possible solutions

- **ii)** mai multe fișiere obișnuite, astfel: `p1.c` și `p2.c`, în subdirectorul `d1` creat anterior; `p3.c` și `p4.cpp`, în subdirectorul `d2` creat anterior; `p5.c` și `p6.h`, în subdirectorul `d4` creat anterior.

  Recomandare: mai întâi, pentru a înțelege exemplele de rezolvare de mai jos, (re)citiți documentațiile comenzilor `touch` și `truncate`, disponibile pe sistemul Linux pe care lucrați folosind comenzile `man 1 touch` și respectiv `man 1 truncate` , sau accesați variantele online ale acestor documentații, disponibile aici și respectiv aici.

Show / Hide some possible solutions

- **iii)** folosind oricare dintre editoarele de texte amintite în lecție, creați un fișier obișnuit numit `15lines.txt`, în subdirectorul `d1` creat anterior, și al cărui conținut să fie secvența numerelor de la 1 la 15, scrise fiecare separat, pe câte o linie de text.

  Show / Hide some possible solutions

- **iv)** în subdirectorul `d2` creat anterior, creați un link hard, numit `hardlnk1`, către fișierul `15lines.txt` creat anterior, precum și un link simbolic, numit `symlnk2`, către același fișier.

  Recomandare: mai întâi, pentru a înțelege exemplele de rezolvare de mai jos, (re)citiți documentația comenzii `ln`, disponibilă pe sistemul Linux pe care lucrați folosind comanda `man 1 ln` , sau accesați o variantă online a acestei documentații, disponibilă aici.

  Show / Hide some possible solutions

  Concluzie: orice fișier stocat pe disc are (cel puțin) un nume: fie exact un nume, fie mai multe nume (create ca link-uri hard către un fișier deja existent, de orice tip ar fi el). Aceste nume sunt sinonime între ele, i.e. poate fi folosit oricare dintre ele pentru a identifica fișierul respectiv. Practic, d.p.d.v. al sistemului de operare, nu există nicio diferență între numele cu care a fost creat fișierul respectiv pe disc și numele suplimentare "adăugate" ulterior acelui fișier prin crearea de link-uri hard către acesta!

3. [Basic file operations #2] **Exemplificare copiere, mutare și ștergere diferite tipuri de fișiere**.
   Notă: în enunțurile de mai jos vom (re)utiliza o parte dintre fișierele și subdirectoarele create în exercițiul anterior!
   Scrieți comenzile care realizează următoarele operații cu fișiere:

   - **i)** copiați subdirectorul `d4`, creat anterior, în subdirectorul `d3` creat anterior; copiați fișierul `p4.cpp`, creat anterior, în subdirectorul `d3` creat anterior, astfel încât copia creată să se numească `p4bis.cpp`.

     Recomandare: mai întâi, pentru a înțelege exemplele de rezolvare de mai jos, (re)citiți documentația comenzii `cp`, disponibilă pe sistemul Linux pe care lucrați folosind comanda `man 1 cp` , sau accesați o variantă online a acestei documentații, disponibilă aici.

     Show / Hide some possible solutions

   - **ii)** mutați fișierul `d1/d3/p4bis.cpp`, creat la pasul anterior, în subdirectorul `d1/d3/d4`, păstrându-i numele nemodificat; redenumiți fișierul `d1/d3/d4/p5.c`, creat la pasul anterior, în `p5.cpp`, fără a-i schimba locația (i.e., directorul în care se află); mutați fișierul `d1/d3/d4/p6.h`, creat la pasul anterior, în subdirectorul `d1/d2`, schimbându-i și numele în `p6bis.h`.

     Recomandare: mai întâi, pentru a înțelege exemplele de rezolvare de mai jos, (re)citiți documentația comenzii `mv`, disponibilă pe sistemul Linux pe care lucrați folosind comanda `man 1 mv` , sau accesați o variantă online a acestei documentații, disponibilă aici.

     Show / Hide some possible solutions

   - **iii)** ștergeți subdirectorul `d1/d3/d4` creat anterior, inclusiv tot conținutul său.

     Recomandare: mai întâi, pentru a înțelege exemplele de rezolvare de mai jos, (re)citiți documentațiile comenzilor `rmdir` și `rm`, disponibile pe sistemul Linux pe care lucrați folosind comenzile `man 1 rmdir` și respectiv `man 1 rm` , sau accesați variantele online ale acestor

documentații, disponibile aici și respectiv aici. Cât despre comanda `cd`, aceasta fiind comandă internă, puteți accesa documentația ei pe sistemul Linux pe care lucrați folosind comanda `help cd` .

Show / Hide some possible solutions

4. [Basic file operations #3] **Exemplificare vizualizarea conținutului fișierelor text, folosind diverse comenzi de vizualizare**.
Notă: în enunțurile de mai jos vom (re)utiliza o parte dintre fișierele și subdirectoarele create în cele două exerciții anterioare!
Scrieți comenzile care realizează următoarele operații cu fișiere:

   - **i)** vizualizați conținutul fișierului `d1/15lines.txt`, creat anterior, afișat în ordinea naturală a liniilor de text și, respectiv, în ordine inversată.

     Recomandare: mai întâi, pentru a înțelege exemplele de rezolvare de mai jos, (re)citiți documentațiile comenzilor cat și tac, disponibile pe sistemul Linux pe care lucrați folosind comenzile `man 1 cat` și respectiv `man 1 tac` , sau accesați variantele online ale acestor documentații, disponibile aici și respectiv aici.

     Show / Hide some possible solutions

   - **ii)** vizualizați fișierul `/etc/passwd`, ce conține informații despre conturile de utilizatori de pe sistemul pe care lucrați, astfel încât afișarea să se facă cu pauză "la trecerea de la un ecran la altul"; repetați această operație, cu posibilitatea de a "naviga" în ambele sensuri.

     Observație: "baza de date" despre conturile de utilizatori, i.e. fișierul `/etc/passwd`, conține linii de text conforme cu următorul șablon:

     username : x : *UID*-ul utilizatorului : *GID*-ul grupului primar de care aparține : *datele personale* ale utilizatorului : *home directory* : *login shell*

     Recomandare: mai întâi, pentru a înțelege exemplele de rezolvare de mai jos, (re)citiți documentațiile comenzilor more și less, disponibile pe sistemul Linux pe care lucrați folosind comenzile `man 1 more` și respectiv `man 1 less` , sau accesați variantele online ale acestor documentații, disponibile aici și respectiv aici.

     Show / Hide some possible solutions

   - **iii)** similar, studiați informațiile despre grupurile de utilizatori de pe sistemul pe care lucrați, consultând fișierul `/etc/group`.

     Observație: "baza de date" despre grupurile de utilizatori, i.e. fișierul `/etc/group`, conține linii de text conforme cu următorul șablon:

     groupname : x : *GID*-ul grupului : o eventuală *listă de username*-uri, ce descrie astfel apartenența utilizatorilor la grupuri secundare

     Show / Hide some possible solutions

- **iv)** vizualizați conținutul fișierului `d1/15lines.txt`, creat anterior, afișând doar primele, respectiv ultimele, N linii din el, valoarea N fiind specificată ca argument al comenzii.

  Recomandare: mai întâi, pentru a înțelege exemplele de rezolvare de mai jos, (re)citiți documentațiile comenzilor head și tail, disponibile pe sistemul Linux pe care lucrați folosind comenzile `man 1 head` și respectiv `man 1 tail` , sau accesați variantele online ale acestor documentații, disponibile aici și respectiv aici.

  Show / Hide some possible solutions

5. [Basic file operations #4] **Exemplificare obținere informații despre fișiere, folosind diverse comenzi**.
   Scrieți comenzile care ne oferă următoarele informații despre fișiere:

   - **i)** Aflați ce fel de conținut au următoarele fișiere (fără să le vizualizați explicit conținutul cu comenzile exemplificate mai sus):
     `/etc/passwd` , `/usr/bin/write` , `/usr/share/man/man1/write.1.gz` , `/usr/include/fcntl.h` și, respectiv, `/etc`.

     Recomandare: mai întâi, pentru a înțelege exemplele de rezolvare de mai jos, (re)citiți documentația comenzii file, disponibilă pe sistemul Linux pe care lucrați folosind comanda `man 1 file` , sau accesați o variantă online a acestei documentații, disponibilă aici.

     Show / Hide some possible solutions

     Important: voi aminti aici doar două dintre opțiunile comenzii `file`, ce sunt utile pentru formatarea output-ului:

     - `-b` sau `--brief` : se folosește pentru a nu mai afișa ca prefix numele fișierului, ci doar tipul de conținut al acestuia
     - `-F` sau `--separator` : se folosește pentru a specifica un alt delimitator (în locul celui implicit ":") ce va separa în output numele fișierului de tipul de conținut al acestuia

   - **ii)** a) Vizualizați metadatele (i.e., proprietățile) asociate fișierului `/etc/passwd`; apoi afișați doar următoarele metadate asociate acestuia: permisiunile în notație simbolică, dimensiunea în octeți, data ultimei modificări și numele lui. b) Obțineți informații similare despre metadatele fișierului `/etc/group`.

     Recomandare: mai întâi, pentru a înțelege exemplele de rezolvare de mai jos, (re)citiți documentația comenzii stat, disponibilă pe sistemul Linux pe care lucrați folosind comanda `man 1 stat` , sau accesați o variantă online a acestei documentații, disponibilă aici.

     Show / Hide some possible solutions

6. [Basic file operations #5] **Exemplificare vizualizare și modificare a informațiilor despre fișiere, folosind diverse comenzi**.
   Vizualizați metadatele (i.e., proprietățile) asociate fișierului `d1/15lines.txt`, creat într-un exemplu anterior (a se revedea exemplul [Basic file operations #1], pct. iii)), iar apoi modificați diverse metadate dintre cele asociate fișierului `d1/15lines.txt`.

Recomandare: mai întâi, pentru a înțelege exemplele de rezolvare de mai jos, (re)citiți documentațiile comenzilor `chmod`, `chown` și `chgrp`, disponibile pe sistemul Linux pe care lucrați folosind comenzile `man 1 chmod` , `man 1 chown` și respectiv `man 1 chgrp` , sau accesați variantele online ale acestor documentații, disponibile aici, aici și respectiv aici.

Show / Hide some examples

7. [cut #1] **Exemplificare selecție de text pe verticală (i.e., pe coloane de text) dintr-un fișier, folosind comanda `cut`.**
Să se scrie comanda care afișează numele de cont și UID-urile tuturor utilizatorilor sistemului.
(Indicație: folosiți comanda cut și informațiile din fișierul `/etc/passwd`.)

Recomandare: mai întâi, pentru a înțelege exemplele de rezolvare de mai jos, (re)citiți documentația comenzii `cut`, disponibilă pe sistemul Linux pe care lucrați folosind comanda `man 1 cut` , sau accesați o variantă online a acestei documentații, disponibilă aici.

Show / Hide some possible solutions Show / Hide a general remark about these possible solutions

Important: mai sus am ilustrat utilizarea a două opțiuni ale comenzii `cut`, utile atunci când "câmpul" (sau "câmpurile") pe care vrem să o/le selectăm din fiecare linie de text a fișierului (sau fișierelor) de prelucrat este/sunt de lungime variabilă, și anume:

- `-f` sau `--fields` : specifică "câmpul" (sau "câmpurile") ce se vor selecta din fiecare linie de text
- `-d` sau `--delimiter` : specifică caracterul folosit ca și delimitator între "câmpuri" consecutive în cadrul liniilor de text.

Voi mai aminti aici alte câteva dintre opțiunile comenzii `cut`, pe care le vom folosi ulterior în exemplele rezolvate din laboratorul #3 pentru a construi diferite lanțuri de comenzi (unele dintre acestea vor utiliza și comanda `cut` pentru a obține funcționalitatea dorită în fiecare caz specific):

- `-b` sau `--bytes` : se folosește pentru a specifica unul sau mai multe "câmpuri" de lungime fixă (nu variabilă), și care încep de la o poziție fixă în cadrul tuturor liniilor de text ce se vor prelucra
- `-c` sau `--characters` : la fel, se folosește pentru a specifica unul sau mai multe "câmpuri" de lungime fixă (nu variabilă), și care încep de la o poziție fixă în cadrul tuturor liniilor de text ce se vor prelucra.
  Notă: dacă sistemul pe care lucrați este configurat să folosească reprezentarea caracterelor pe 1 octet, atunci nu este nicio diferență între efectele folosirii opțiunilor `-c` sau `-b` .
- `--complement` : afișează restul din fiecare linie de text, excluzând "câmpul" (sau "câmpurile") specificat/e
- `--output-delimiter` : înlocuiește, în outputul rezultat, caracterul delimitator între "câmpuri" cu un alt caracter (sau chiar cu un cuvânt, i.e. o secvență de mai multe caractere)

8. [grep #1] **Exemplificare selecție de text pe orizontală (i.e., pe linii de text) dintr-un fișier, folosind comanda `grep`.**
Să se scrie comanda care selectează din fișierul `/etc/passwd` doar informațiile despre un utilizator specificat ca argument al comenzii.
(Indicație: folosiți comanda grep.)

Recomandare: mai întâi, pentru a înțelege exemplul de rezolvare de mai jos, (re)citiți documentația comenzii `grep`, disponibilă pe sistemul Linux pe care lucrați folosind comanda `man 1 grep` , sau accesați o variantă online a acestei documentații, disponibilă aici.

Show / Hide the solution

Important: voi aminti aici doar câteva dintre opțiunile comenzii `grep`, pe care le vom folosi ulterior în exemplele rezolvate din laboratorul #3 pentru a construi diferite lanțuri de comenzi (unele dintre acestea vor utiliza și comanda `grep` pentru a obține funcționalitatea dorită în fiecare caz specific):

- `-c` sau `--count` : în loc de outputul obișnuit (i.e., afișarea tuturor liniilor care conțin șablonul specificat), afișează doar numărul de linii ce conțin șablonul specificat
- `-w` sau `--word-regexp` : afișează doar liniile care conțin șablonul specificat ca și cuvânt "întreg", i.e. nu este sub-cuvânt într-un cuvânt mai mare.
  Notă: prin definiție, un "cuvânt" este o secvență de caractere care sunt litere, cifre sau '_' .
- `-v` sau `--invert-match` : afișează liniile care NU conțin șablonul specificat
- `-i` sau `--ignore-case` : face *pattern-matching case-insensitive* (i.e., ignoră diferența dintre litere majuscule și minuscule, atât în șablonul specificat, cât și în conținutul fișierului de prelucrat)
- `-o` sau `--only-matching` : nu afișează în întregime fiecare linie de text ce conține șablonul specificat, ci doar partea (nevidă) a liniei ce conține exact șablonul specificat
  Notă: este utilă mai ales atunci când șablonul specificat este o expresie regulată, i.e. descrie un set de cuvinte.
- `-E` sau `--extended-regexp` : interpretează șablonul specificat ca o *expresie regulată extinsă*
  Notă: în mod implicit, sau cu `-G` sau `--basic-regexp`, acesta este interpretat ca o *expresie regulată simplă*.
- `-F` sau `--fixed-strings` : interpretează șablonul specificat ca un cuvânt "constant", nu ca o expresie regulată
  Notă: în mod implicit, acesta este interpretat ca o *expresie regulată simplă*.
- `-q` sau `--quiet` sau `--silent` : nu afișează nimic pe `stdout`, ci returnează imediat codul de terminare 0 la prima linie întâlnită care conține șablonul specificat.

9. [sort #1] **Exemplificare sortarea conținutului unui fișier, folosind comanda sort**.
   Să se scrie comanda care să sorteze descrescător liniile de text din fișierul `/etc/passwd`, iar rezultatul sortării să fie salvat în fișierul `output.txt`.
   (Indicație: folosiți comanda sort cu cele două opțiuni corespunzătoare cerințelor din enunț.)

   Recomandare: mai întâi, pentru a înțelege exemplele de rezolvare de mai jos, (re)citiți documentația comenzii `sort`, disponibilă pe sistemul Linux pe care lucrați folosind comanda `man 1 sort` , sau accesați o variantă online a acestei documentații, disponibilă aici.

   Show / Hide some possible solutions

10. [sort #2] **Exemplificare sortarea conținutului unui fișier, folosind comanda sort**.
    Să se scrie comanda care să sorteze crescător după UID liniile de text din fișierul `/etc/passwd`.
    (Indicație: folosiți comanda sort cu trei opțiuni: opțiunea pentru sortare numerică, opțiunea pentru sortare după cheia câmpul 3 și opțiunea pentru stabilirea separatorului dintre câmpuri.)

    Show / Hide some possible solutions

11. [wc #1] **Exemplificare numărarea unităților de text din conținutul unui fișier, folosind comanda wc**.
Să se scrie comanda care să ne indice câte conturi de utilizatori și, respectiv, câte grupuri de utilizatori există pe sistemul pe care lucrați.
(Indicație: folosiți comanda wc aplicată celor două baze de date, ce conțin utilizatorii sistemului și, respectiv, grupurile de utilizatori.)

Recomandare: mai întâi, pentru a înțelege exemplele de rezolvare de mai jos, (re)citiți documentația comenzii `wc`, disponibilă pe sistemul Linux pe care lucrați folosind comanda `man 1 wc` , sau accesați o variantă online a acestei documentații, disponibilă aici.

Show / Hide the solution

12. [find #1] **Exemplificare căutare de fișiere, folosind comanda find**.
Să se scrie comanda care să parcurgă recursiv directorul dat ca argument și să afișeze permisiunile tuturor fișierelor și subdirectoarelor întâlnite.
(Indicație: folosiți comanda find și cu acțiunea de apelare a comenzii stat, sau acțiunea echivalentă de afișare cu format.)

Recomandare: mai întâi, pentru a înțelege exemplele de rezolvare de mai jos, (re)citiți documentația comenzii `find`, disponibilă pe sistemul Linux pe care lucrați folosind comanda `man 1 find` , sau accesați o variantă online a acestei documentații, disponibilă aici.

Show / Hide some possible solutions

Topici avansate (*pentru studenții mai buni, pasionați de tehnologie, ce doresc să afle informații și despre alte subiecte suplimentare, în plus față de setul minimal de cunoștințe predat în cadrul disciplinei SO*) :
Pentru a găsi și înțelege răspunsul la întrebarea formulată în cadrul rezolvării exercițiului de mai sus, vă recomand să citiți explicațiile suplimentare prezentate aici.

13. [find #2] **Exemplificare căutare de fișiere, folosind comanda find**.
Să se scrie comanda ce afișează pentru toate fișierele găsite în directorul dat ca argument în linia de comandă, parcurs recursiv, fișiere ale căror nume încep sau se termină cu litera 'a', următoarele proprietăți: numele fișierului, data ultimei modificări și dimensiunea fișierului în Kbytes.
(Indicație: folosiți comanda find și cu opțiunea de combinare alternativă a mai multor criterii de căutare după nume, iar la partea de acțiuni cu opțiunea -printf cu parametrii adecvați.)

Show / Hide some possible solutions